Industrial PPE & Safety Compliance AI System

Industrial PPE & Safety Compliance AI System

Project Documentation — BPUT Hackathon 2026

---

1. Problem Statement

Factories and industrial sites need continuous monitoring for worker safety compliance (helmets, gloves, vests, fire hazards) and existing solutions fail because they:

- Only give binary “violation / no violation” alerts with no context
- Break down under real factory conditions (dust, steam, poor lighting, bad camera angles)
- Flood supervisors with false positives, causing “alert fatigue”
- Are cloud-dependent, slow, and expensive to run across many camera feeds
- Aren’t built with the actual end-user (the floor supervisor) in mind

Goal: Build a deployable, edge-capable, multi-camera safety intelligence system — not just a demo that detects a helmet in one clean video.

---

2. Core Features (Must-Build)

These are the non-negotiable features needed to be considered “industry-ready” rather than a hackathon demo.

2.1 Contextual Violation Detection

- Detect specific PPE items individually: helmet, gloves, safety vest, goggles, boots, mask
- Classify *which* item is missing per person, not just “violation detected”
- Track violations per-worker across frames (avoid duplicate alerts for the same person/event)

2.2 Severity-Based Alert Prioritization

Build a tiered alert logic system, for example:

Severity	Example Trigger	Response
Critical	Fire / smoke detected, gas leak zone breach	Immediate alarm + notify safety officer instantly
High	Missing helmet in active machinery	Real-time alert to

	zone	supervisor
Medium	Missing gloves/vest in low-risk zone	Logged + periodic compliance digest
Low	Minor posture/positioning issues	Dashboard log only

2.3 Supervisor Dashboard (Human-in-the-Loop)

- Live view across multiple camera streams
- Each alert shows: location/zone, camera ID, violation type, severity, timestamp, snapshot/clip
- Acknowledge / escalate / dismiss workflow for supervisors
- Daily/weekly compliance summary reports per zone or shift

2.4 Multi-Stream Processing

- Architecture must support N simultaneous camera feeds, not just one demo video
- Per-stream status indicators (online/offline/degraded)
- Load-aware processing so streams don't block each other

2.5 Edge-First Processing

- Run inference locally (e.g., on a Jetson/NUC-class device or on-prem server) instead of routing all video to the cloud
- Only send metadata/alerts/thumbnails to the cloud/dashboard, not raw video, to save bandwidth
- Justify this choice in your docs: latency + bandwidth + cost

---

3. Robustness Requirements (Real-World Engineering)

Challenge	What Judges Will Look For
Dust / steam / smoke in frame	Model tested against degraded visibility, not just clean footage
Poor lighting (night shift, backlit)	Augmented training data covering low-light/overexposed conditions
Non-ideal camera angles	Training data includes oblique/high-mounted angles, not just front-facing
False positives	Explicit example: bright orange/yellow clothing must NOT trigger a fire alert
Occlusion (workers overlapping, partially hidden)	Tracking logic that avoids duplicate/missed detections

Documentation tip: Include a short "failure case gallery" — screenshots of tricky scenarios and how your system handled them. This alone differentiates you from 90% of teams.

---

4. Extra / Differentiating Features (Should-Build if Time Allows)

These push the project from “good” to “stands out to judges.”

- **Zone-based rules engine** — different PPE requirements per zone (e.g., welding bay requires goggles + gloves, warehouse only requires helmet)
- **Worker ID / badge integration** — link violations to a worker ID (via badge/RFID, not facial recognition, to avoid privacy issues) for accountability trends
- **Trend analytics** — “Zone B has 3x more glove violations on night shift” style insights, not just raw alerts
- **Auto-generated compliance reports** — exportable PDF/CSV for safety audits
- **Alert routing/escalation** — if unacknowledged in X minutes, auto-escalate to next level (SMS/WhatsApp/email integration)
- **Offline-first resilience** — buffer alerts locally if network drops, sync when back online
- **Cost/ROI calculator built into the pitch** — camera cost + edge device cost vs. estimated reduction in incidents/insurance premiums
- **Model confidence display** — show detection confidence score so supervisors can judge borderline cases themselves

---

5. Suggested Technical Architecture

```
[Camera Streams]
      │
      ▼
[Edge Device: PPE Detection Model (YOLOv8/YOLO-NAS or similar)]
      │  (local inference, only metadata + alert leaves device)
      ▼
[Alert Logic Engine] → severity classification, dedup, zone rules
      │
      ▼
[Backend API] → stores events, manages escalation, serves dashboard
      │
      ▼
[Supervisor Dashboard] → live streams, alert feed, analytics,
reports
```

Suggested stack (adjust to your team’s strengths): - Detection: YOLOv8/YOLO-NAS (fine-tuned on PPE datasets + your own augmented/edge-case data) - Edge runtime: ONNX/TensorRT for optimized local inference - Backend: FastAPI/Node.js + a lightweight message queue for multi-stream handling - Dashboard: React + WebSocket for live alert feed - Storage: PostgreSQL (structured events) + object storage for snapshots/clips

---

6. What to Avoid (“The Tool Trap”)

- Don’t just call a pre-trained detection API and call it done — explain *why* your approach handles gaps existing tools don’t
- Don’t over-index on hackathon-only test videos — research real industry benchmarks/datasets
- Don’t optimize for judges’ first impression only — design as if the end-user is a floor supervisor who will use this daily

---

7. Build & Iteration Plan

1. **Day 1:** Data collection/augmentation (include dusty, dark, angled,

and false-positive-prone samples) + baseline detection model
2. **Day 1–2:** Severity/alert logic layer + zone rules
3. **Day 2:** Multi-stream handling + edge optimization pass
4. **Day 2–3:** Dashboard build (live feed, alert cards, acknowledge workflow)
5. **Day 3:** Stress-test against corner cases; tune false positive/negative rate
6. **Final:** Build the pitch around real-world deployment cost, impact on supervisor workflow, and the specific gap you're filling — not just the demo

---

8. Judging Alignment Checklist

- ☐ Detects specific PPE items, not just binary violation
- ☐ Has a working severity/priority system (fire ≠ missing glove)
- ☐ Dashboard shows location, type, severity, and timestamp per alert
- ☐ Demonstrated robustness against dust/steam/lighting/angle issues
- ☐ At least one documented false-positive corner case handled (e.g., bright clothing ≠ fire)
- ☐ Supports multiple simultaneous camera streams
- ☐ Uses edge/local inference with a stated bandwidth/latency justification
- ☐ Includes a cost/deployment feasibility angle
- ☐ Clear articulation of the real industry gap being solved (not just "we used AI")